
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 3

Desenvolvimento Estruturado de Programas em C

Fazendo a Diferença

Exercícios 3.47 – 3.49

Crescimento populacional, frequência cardíaca, criptografia

Contents

1	Exercício 3.47 – Calculadora de Crescimento Populacional Mundial	2
2	Exercício 3.48 – Calculadora de Frequência Cardíaca	4
3	Exercício 3.49 – Criptografia de 4 Dígitos	7

1. Exercício 3.47 – Calculadora de Crescimento Populacional Mundial

Enunciado 3.47

Use a Web para determinar a população mundial atual e sua taxa de crescimento anual. Escreva uma aplicação que leia esses valores, depois apresente a população mundial estimada após um, dois, três, quatro e cinco anos.

Pesquisa: População Mundial e Taxa de Crescimento

Pesquisa externa realizada

Segundo dados das Nações Unidas (UN/DESA), a população mundial em 2024 era de aproximadamente **8,1 bilhões de pessoas**, com uma taxa de crescimento anual de cerca de **0,87%** (aproximadamente 70 milhões de pessoas a mais por ano).

Fórmula de projeção populacional (juros compostos aplicada a populações):

$$P_n = P_0 \cdot (1 + r)^n$$

onde P_n é a população após n anos, P_0 é a população atual, e r é a taxa de crescimento anual (em decimal: 0,0087 para 0,87%).

Solução em C

Resposta detalhada

```
1  /* Ex3.47: crescimento_populacional.c
2     Estima populacao mundial nos proximos 5 anos */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      float populacao;      /* populacao mundial atual */
8      float taxa;           /* taxa de crescimento anual
9                             (decimal) */
10     float projecao;        /* populacao projetada */
11     int anos = 1;          /* contador de anos */
12
13     printf( "Digite a populacao mundial atual (em
14             bilhoes): " );
15     scanf( "%f", &populacao );
16
17     printf( "Digite a taxa anual de crescimento (
18             decimal, "
19             "ex: 0.0087 para 0.87%%): " );
20     scanf( "%f", &taxa );
```

```
18
19     printf( "\n=== PROJECAO POPULACIONAL ===\n" );
20     printf( "Ano atual: %.4f bilhoes\n", populacao );
21
22     projecao = populacao;
23     while ( anos <= 5 ) {
24         projecao = projecao * ( 1.0f + taxa );
25         printf( "Apos %d ano(s): %.4f bilhoes\n", anos
26             , projecao );
27         ++anos;
28     }
29
30     return 0;
31 } /* fim da funcao main */
```

Saída do programa

```
Digite a populacao mundial atual (em bilhoes): 8.1
Digite a taxa anual de crescimento (decimal, ex: 0.0087 para
0.87%): 0.0087

=== PROJECAO POPULACIONAL ===
Ano atual: 8.1000 bilhoes
Apos 1 ano(s): 8.1705 bilhoes
Apos 2 ano(s): 8.2416 bilhoes
Apos 3 ano(s): 8.3133 bilhoes
Apos 4 ano(s): 8.3856 bilhoes
Apos 5 ano(s): 8.4585 bilhoes
```

Reflexão: O crescimento composto faz com que pequenas taxas anuais gerem grandes mudanças ao longo do tempo. Em apenas 5 anos, a população mundial projetada cresce cerca de **358 milhões de pessoas** – o equivalente à população dos EUA! Esse exemplo ilustra a importância de discussões sobre sustentabilidade, recursos naturais e planejamento global.

Nota: O %% dentro da string de printf é a forma de exibir o caractere % literalmente.

2. Exercício 3.48 – Calculadora de Frequência Cardíaca

Enunciado 3.48

Segundo a American Heart Association (AHA), a fórmula para calcular a frequência cardíaca máxima é **220 menos a idade em anos**. A frequência cardíaca-alvo durante exercícios deve estar entre **50% e 85%** da máxima. Crie um programa que leia a data de nascimento e a data atual (dia, mês, ano de cada), calcule e exiba a idade da pessoa, sua frequência cardíaca máxima e a faixa-alvo.

Pesquisa: Cálculo da Idade

Pesquisa externa realizada

Cálculo correto da idade em anos completos:

- Diferença básica: $\text{idade} = \text{ano_atual} - \text{ano_nascimento}$
- Ajuste: se o aniversário ainda *não* ocorreu este ano, subtrair 1 da idade. O aniversário ocorreu se: $\text{mês_atual} > \text{mês_nascimento}$, OU $(\text{mês_atual} == \text{mês_nascimento} \text{ E } \text{dia_atual} \geq \text{dia_nascimento})$.

Fórmulas da AHA:

- Frequência máxima: $FC_{\text{máx}} = 220 - \text{idade}$
- Faixa-alvo: 50% a 85% da máxima

Solução em C

Resposta detalhada

```
1  /* Ex3.48: frequencia_cardiaca.c
2     Calcula idade, FC maxima e faixa-alvo durante
3     exercicios */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9     int diaNasc, mesNasc, anoNasc; /* data de
10     nascimento */
11     int diaAtual, mesAtual, anoAtual; /* data atual
12     */
13     int idade;
14     int fcMaxima;
15     float fcMinima, fcAlvo; /* limites inferior e
16     superior em bpm */
17
18     /* Entrada: data de nascimento */
19     printf( "=== Calculadora de Frequencia Cardiaca
20     ===\n\n" );
21     printf( "DATA DE NASCIMENTO\n" );
22     printf( "Dia: " );      scanf( "%d", &diaNasc );
```

```
17     printf( "Mes: " );           scanf( "%d", &mesNasc );
18     printf( "Ano: " );           scanf( "%d", &anoNasc );
19
20     /* Entrada: data atual */
21     printf( "\nDATA ATUAL\n" );
22     printf( "Dia: " );           scanf( "%d", &diaAtual );
23     printf( "Mes: " );           scanf( "%d", &mesAtual );
24     printf( "Ano: " );           scanf( "%d", &anoAtual );
25
26     /* Calcula a idade em anos completos */
27     idade = anoAtual - anoNasc;
28     /* Se o aniversario ainda nao chegou neste ano,
29        subtrai 1 */
29     if ( mesAtual < mesNasc ) {
30         idade = idade - 1;
31     }
32     else if ( mesAtual == mesNasc && diaAtual <
33              diaNasc ) {
34         idade = idade - 1;
35     }
36
37     /* Calcula FC maxima e faixa-alvo */
38     fcMaxima = 220 - idade;
39     fcMinima = fcMaxima * 0.50f; /* 50% da maxima */
40     fcAlvo    = fcMaxima * 0.85f; /* 85% da maxima */
41
42     /* Exibe os resultados */
43     printf( "\n== RESULTADO ==\n" );
44     printf( "Idade: %d anos\n", idade );
45     printf( "Frequencia cardiaca maxima: %d bpm\n",
46            fcMaxima );
47     printf( "Faixa-alvo durante exercicios: "
48            "%.1f a %.1f bpm\n", fcMinima, fcAlvo );
49
50     printf( "\nATENCAO: Sempre consulte um medico
51            antes de iniciar\n" );
52     printf( "                ou modificar um programa de
53            exercicios.\n" );
54
55     return 0;
56 } /* fim da funcao main */
```

Saída do programa

```
=== Calculadora de Frequencia Cardiaca ===  
DATA DE NASCIMENTO  
Dia: 15  
Mes: 8  
Ano: 1995  
DATA ATUAL  
Dia: 5  
Mes: 5  
Ano: 2026  
=== RESULTADO ===  
Idade: 30 anos  
Frequencia cardiaca maxima: 190 bpm  
Faixa-alvo durante exercicios: 95.0 a 161.5 bpm  
ATENCAO: Sempre consulte um medico antes de iniciar  
          ou modificar um programa de exercicios.
```

Atenção

A fórmula $220 - \text{idade}$ é uma *estimativa simplificada* da AHA. Pesquisas mais recentes (Tanaka et al., 2001) sugerem fórmulas mais precisas, como $FC_{\text{máx}} = 208 - 0,7 \times \text{idade}$. As frequências reais variam com saúde, condição física e sexo. Consulte sempre um médico.

3. Exercício 3.49 – Criptografia de 4 Dígitos

Enunciado 3.49

Uma empresa transmite dados como inteiros de quatro dígitos. Para codificar:

- Substitua cada dígito por: $(d_{\text{dígito}} + 7) \% 10$
- Troque o 1º com o 3º dígito; troque o 2º com o 4º dígito.

Escreva um programa que codifique um inteiro de 4 dígitos. Escreva outro programa que decodifique (revertendo o esquema).

Análise do Algoritmo

Codificação: se o dígito é d , o novo dígito é $(d + 7) \bmod 10$.

Exemplo com o número **1234**:

- Dígitos originais: $d_1 = 1, d_2 = 2, d_3 = 3, d_4 = 4$
- Após $(d + 7) \bmod 10$: $d_1 = 8, d_2 = 9, d_3 = 0, d_4 = 1$
- Após troca ($1^{\circ}3^{\circ}, 2^{\circ}4^{\circ}$): $d_1 = 0, d_2 = 1, d_3 = 8, d_4 = 9$
- Número codificado: **0189**

Decodificação: reverter ambas as operações.

- Inversa da troca: trocar novamente $1^{\circ}3^{\circ}$ e $2^{\circ}4^{\circ}$ (a troca é sua própria inversa).
- Inversa de $(d + 7) \bmod 10$: $(d + 3) \bmod 10$ (pois $7 + 3 = 10 \equiv 0$).

Programa 1 – Codificador

Resposta detalhada

```

1  /* Ex3.49(a): codificador.c
2     Codifica um inteiro de 4 dígitos */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int numero;
8      int d1, d2, d3, d4;          /* dígitos originais
9                                     */
10     int e1, e2, e3, e4;          /* dígitos apos (+7) mod
11                                     10 */
12     int codificado;
13
14     printf( "Digite um inteiro de 4 dígitos: " );
15     scanf( "%d", &numero );
16
17     /* Extrai os 4 dígitos */
18     d1 = numero / 1000;
19     d2 = ( numero / 100 ) % 10;
20     d3 = ( numero / 10 ) % 10;
21     d4 = numero % 10;

```

```
21      /* Aplica (digito + 7) mod 10 */
22      e1 = ( d1 + 7 ) % 10;
23      e2 = ( d2 + 7 ) % 10;
24      e3 = ( d3 + 7 ) % 10;
25      e4 = ( d4 + 7 ) % 10;
26
27      /* Troca: 1o <-> 3o, 2o <-> 4o */
28      /* O codificado fica na ordem: e3, e4, e1, e2 */
29      codificado = e3 * 1000 + e4 * 100 + e1 * 10 + e2;
30
31      printf( "Numero codificado: %04d\n", codificado );
32
33      return 0;
34 } /* fim da funcao main */
```

Nota: O especificador `%04d` garante que o número seja exibido com 4 dígitos, preenchendo com zeros à esquerda se necessário (ex.: 189 → 0189).

Programa 2 – Decodificador

Resposta detalhada

```
1  /* Ex3.49(b): decodificador.c
2     Decodifica um inteiro de 4 dígitos previamente
3     codificado */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int numero;
10     int c1, c2, c3, c4;      /* dígitos do numero
11                               codificado */
12     int d1, d2, d3, d4;      /* dígitos originais
13                               */
14     int decodificado;
15
16     printf( "Digite um inteiro codificado de 4 dígitos
17             : " );
18     scanf( "%d", &numero );
19
20     /* Extrai os 4 dígitos do codificado */
21     c1 = numero / 1000;
22     c2 = ( numero / 100 ) % 10;
23     c3 = ( numero / 10 ) % 10;
24     c4 = numero % 10;
25
26     /* Reverte a troca: 1o <-> 3o, 2o <-> 4o */
27     /* Após reverter: posicoes ficam c3, c4, c1, c2 */
28 }
```

```
24      /* Reverte (d+7) mod 10 aplicando (c+3) mod 10 */
25      d1 = ( c3 + 3 ) % 10;
26      d2 = ( c4 + 3 ) % 10;
27      d3 = ( c1 + 3 ) % 10;
28      d4 = ( c2 + 3 ) % 10;
29
30      decodificado = d1 * 1000 + d2 * 100 + d3 * 10 + d4
31      ;
32
33      printf( "Numero decodificado: %04d\n",
34              decodificado );
35
36      return 0;
37 } /* fim da funcao main */
```

Saída do programa

```
Digite um inteiro codificado de 4 digitos: 189
Numero decodificado: 1234
```

Verificação Matemática

Resposta detalhada

Por que $(c + 3) \bmod 10$ inverte $(d + 7) \bmod 10$?

Da operação direta: $c = (d + 7) \bmod 10$.

Aplicando a inversa: $(c + 3) \bmod 10 = ((d + 7) + 3) \bmod 10 = (d + 10) \bmod 10 = d \bmod 10 = d$.

Como d é um dígito (0 a 9), $d \bmod 10 = d$. Logo, recuperamos o dígito original.

Verificação completa com $1234 \rightarrow 0189 \rightarrow 1234$:

Codificação 1234:

- Dígitos: 1, 2, 3, 4
- Após $+7 \bmod 10$: 8, 9, 0, 1
- Após troca: 0, 1, 8, 9 $\rightarrow$ número **0189**

Decodificação 0189:

- Dígitos: 0, 1, 8, 9
- Reverte troca: 8, 9, 0, 1
- Após $+3 \bmod 10$: $(8 + 3) \% 10 = 1$, $(9 + 3) \% 10 = 2$, $(0 + 3) \% 10 = 3$, $(1 + 3) \% 10 = 4$
- Resultado: 1, 2, 3, 4 $\rightarrow$ número **1234**

Reflexão sobre Criptografia

Da brincadeira didática à criptografia real

O esquema deste exercício é uma forma muito simples de criptografia, conhecida como **cifra de substituição com transposição**. Ele tem grandes limitações:

- **Reversibilidade óbvia:** Quem conhece o algoritmo decodifica trivialmente.
- **Sem chave secreta:** Não há “segredo” que separe usuários legítimos de atacantes.
- **Espaço minúsculo de mensagens:** Apenas 10.000 valores possíveis (0–9999).

Sistemas modernos usam:

- **Criptografia simétrica** (AES) – chave única compartilhada
- **Criptografia de chave pública** (RSA, PGP) – chave pública para cifrar e chave privada para decifrar; baseada em problemas matemáticos computacionalmente difíceis (fatoração de inteiros grandes, logaritmo discreto)
- **Funções hash criptográficas** (SHA-256) – garantem integridade dos dados

Esse exercício é uma porta de entrada conceitual para um campo fundamental da ciência da computação contemporânea: a **segurança da informação**.

Resumo – Fazendo a Diferença no Capítulo 3

Os três exercícios e suas conexões

3.47 – Crescimento Populacional: Aplicação de cálculos compostos em escala global. O laço `while` permite projetar cenários ano a ano. Conecta-se ao Ex. 1.10 (carbono) e ao Ex. 2.33 (transporte solidário) – todos lidam com sustentabilidade e impacto humano no planeta.

3.48 – Frequência Cardíaca: Aplicação direta de `if...else` para o cálculo cuidadoso da idade, e da estrutura sequencial para determinar limites cardiovasculares seguros. Conecta-se ao Ex. 1.11 e ao Ex. 2.32 (IMC) – a programação como ferramenta para a saúde.

3.49 – Criptografia: Manipulação avançada de dígitos com aritmética modular. Conecta-se aos Exercícios 2.30 (separar dígitos) e 3.35 (palíndromo) – a aritmética inteira como ferramenta para resolver problemas surpreendentemente complexos.

Boa prática de programação

Os três exercícios deste capítulo demonstram como, com apenas as ferramentas dos Capítulos 1, 2 e 3 – variáveis, `if/else`, `while` e operações aritméticas – já é possível abordar problemas reais e relevantes. À medida que novos recursos da linguagem forem introduzidos nos próximos capítulos, sua capacidade de resolver problemas crescerá exponencialmente.